SECTION 1

Voting Classifiers

- **Wisdom of crowds:** Aggregating many moderately accurate models can outperform the best individual predictor when errors differ.
- **Hard voting:** `VotingClassifier` picks the majority class; diversity matters because correlated mistakes hurt the ensemble.
- **Soft voting:** Averages class probabilities when all estimators expose `predict_proba`, often improving accuracy over hard votes.
- **Heterogeneous bases:** Mixing algorithms (e.g., logistic regression, SVM, random forest) increases decorrelated errors.
- **SVM probabilities:** `probability=True` enables soft voting at the cost of slower calibration via internal cross-validation.

SECTION 2

Bagging and Pasting

- **Bootstrap aggregating:** `BaggingClassifier` trains many models on random subsets with replacement of the training set.
- **Pasting:** Sampling without replacement is slightly less diverse but still reduces variance through aggregation.
- **Bias–variance trade-off:** Individual bagged models are higher bias alone; averaging lowers variance with similar bias.
- **Parallel training:** Bagging and pasting scale well because independent fits can run on multiple CPU cores (`n_jobs`).
- **Out-of-bag evaluation:** With bagging, roughly 63% of points appear in each bootstrap sample; `oob_score=True` estimates generalization without a hold-out set.

SECTION 3

Random Patches and Random Subspaces

- **Feature subsampling:** `max_features` and `bootstrap_features` train each base learner on random feature subsets.
- **Random patches:** Subsample both instances and features, useful for high-dimensional inputs such as images.
- **Random subspaces:** Keep all instances but vary feature columns, increasing diversity among trees or other bases.

SECTION 4

Random Forests and Extra-Trees

- **`RandomForestClassifier`:** An ensemble of decision trees with bagging by default and tuned tree hyperparameters.
- **Random split search:** At each node only a random subset of features is evaluated, trading a bit of bias for lower correlation among trees.
- **Extra-trees:** `ExtraTreesClassifier` adds random thresholds, training faster with more randomness; performance must be compared empirically.
- **Feature importances:** Mean impurity decrease per feature, weighted by node samples, ranks inputs after training (`feature_importances_`).

SECTION 5

Boosting

- **Sequential correction:** Boosting adds learners that focus on remaining errors of the ensemble, unlike parallel bagging.
- **AdaBoost:** Reweights misclassified points; predictors receive weights α from their errors; `SAMME.R` uses probabilities when available.
- **Learning rate:** Scales how aggressively each round emphasizes hard examples; too high a rate can overfit.
- **Gradient boosting:** New regressors fit residuals of prior trees; `GradientBoostingRegressor` supports shrinkage via `learning_rate` and tree depth limits.
- **Early stopping:** `staged_predict`, validation curves, or `warm_start` help pick the number of trees without overfitting.
- **Stochastic boosting:** `subsample < 1` trains each tree on a fraction of data for speed and variance reduction.
- **XGBoost:** External library cited for speed, regularization, and built-in early stopping on validation sets.

SECTION 6

Stacking

- **Meta-learner:** A blender model learns to combine base predictions instead of simple voting.
- **Hold-out training:** First-layer models train on one split; their out-of-sample predictions become features to train the blender.
- **Multilayer stacks:** Extra splits can train deeper layers of blenders for richer ensembles.
- **Scikit-Learn gap:** Stacking is not a single built-in class; custom pipelines or libraries such as DESlib implement the pattern.